

Sisteme de Operare

Cap 1. Scopul sistemelor de operare

February 20, 2025

- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului
- 7 Open-source
- 8 Bibliografie

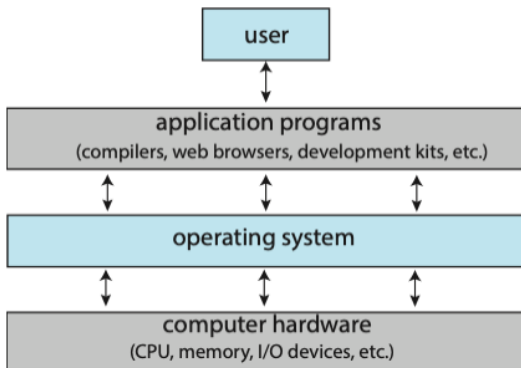
- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului
- 7 Open-source
- 8 Bibliografie

Ce este un sistem de operare

- comandă hardware-ul sistem (computerul), este intermediarul dintre utilizator și hardware
- sunt peste tot (auto, telefoane mobile, IoT, cloud)
- sistem complex, care dispune de arhitectura calculatorului: CPU, memorie, dispozitive I/O, dispozitive de stocare
- vom descrie organizarea unui sistem de calcul, precum și rolul întreruperilor, componentele unui sistem multiprocesor, tranziția de la user mode la kernel mode, precum și diverse utilizări

Ce face un sistem de operare

- calculatorul: hardware, SO, programe aplicații, utilizator
- controlează hardware-ul și coordonează folosirea resurselor (CPU, memorie etc.) de către diverse aplicații și utilizatori
- furnizează mediul în care programele funcționează



Interacțiunea SO cu utilizatorul

- PC: monitor, tastatură, mouse: utilizatorul monopolizează resursele
- ne interesează maximizarea muncii (jocului) realizat de utilizator, ușurința în utilizare, și nu securitatea sau utilizarea resurselor
- utilizatorii interacționează cu telefoane mobile sau tablete, conectate în rețele wireless; ele dispun de touch screen, sau chiar prin voice recognition (Siri)
- calculatoarele embedded din dispozitivele casnice sau automobile aproape nu au interfețe utilizator; deși au uneori LED-uri de status și tastaturi numerice, sunt proiectate să ruleze fără intervenția utilizatorului

Interacțiunea SO cu hardware-ul

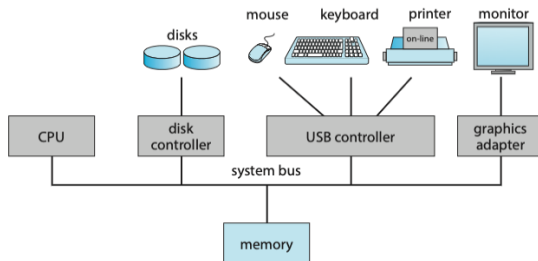
- sistemul de operare interacționează direct cu hardware-ul; programul aplicație folosește serviciile puse la dispoziție de SO
- SO este un alocator de resurse: timp CPU, spațiu de memorie, spațiu de stocare, dispozitive I/O; SO decide cum se alocă aceste resurse și arbitrează conflictele de acces la ele
- SO este un program de control, urmărește execuția programelor utilizator pentru a preveni erorile și folosirea improprie a mașinii de calcul

Istoric și noțiuni

- 1940 - calculatoare pentru scopuri unice: calculul traiectoriilor balistice, spargerea codurilor, calculul taxelor
- 1960 - legea lui Moore: numărul de tranzistoare al unui circuit integrat se dublează la fiecare 18 luni
- calculatoarele au crescut ca funcționalitate și au scăzut în dimensiuni
- **system kernel**: programul care rulează în orice moment pe calculator
- pe lângă kernel avem **programele sistem**, asociate cu sistemul de operare dar care nu sunt kernel și **programele aplicație**, care sunt programe neasociate cu sistemul de operare
- 1988 - US DoJ a dat în judecată MS pentru că acesta a inclus în sistemul de operare prea multă funcționalitate, împiedicând competiția între vânzătorii de aplicații (ex. MS Internet Explorer)
- SO mobile includ nu doar kernel-ul ci și **middleware**, un set framework-uri care furnizează servicii suplimentare dezvoltatorilor (primitive multimedia, grafică, baze de date)

- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului
- 7 Open-source
- 8 Bibliografie

Organizarea unui calculator

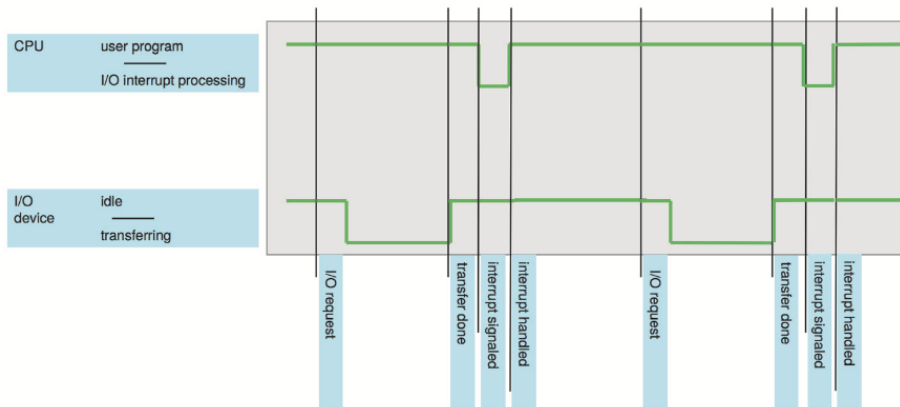


- sistemul de calcul constă din 1 sau mai multe CPU-uri (procesoare) legate cu o magistrală (**bus**) de o serie de controllere de dispozitive
- în funcție de controller, mai multe dispozitive de acel fel pot fi conectate (ex. USB - Universal Serial Bus)
- controllerele au: subset de registre și zonă de memorie buffer locală
- pentru fiecare controller SO are un **device driver**, o bucată de cod (software) scris specific pentru acel controller, furnizând SO interfețe omogene spre dispozitiv; concurează la memorie în paralel cu CPU

Întreruperi

- device driver-ul programează în regiștrii locali citirea/scrierea datelor; controller-ul dispozitivului examinează regiștrii și pornește transferul de date către buffer-ul local; odată transferul terminat, device driver-ul este informat de aceasta de către controller; device driver-ul va da controlul sistemului de operare, informând că datele se află la o anumită adresă
- informarea controller-ului către device driver vine asincron, în timp ce SO face altceva, prin intermediul unei **întreruperi**
- semnalul de întrerupere se trimite pe bus-ul sistem

Timeline-ul unei întreruperi

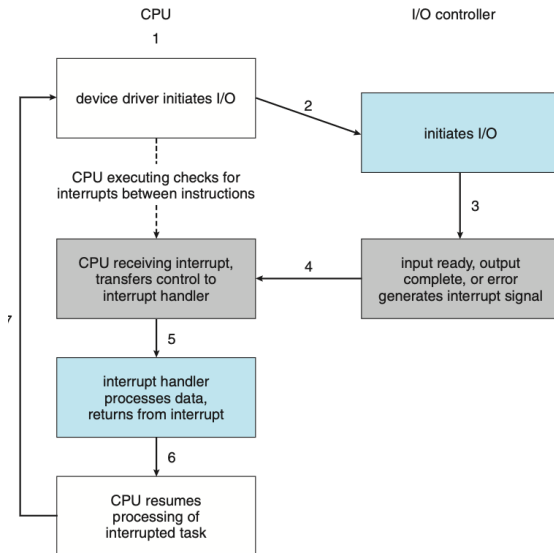


- la apariția întreruperii, controlul se dă la o rutină specifică de tratare
- rutina trebuie să proceseze repede, întreruperile apar des
- UNIX, Windows mențin un **vector de întrerupere**, o tabelă de adrese de rutine de întrerupere, pentru diverse dispozitive

Implementarea întreruperii

- CPU are o linie electrică **interrupt request line** (IRQ)
- la detecția unui front activ, CPU citește numărul întreruperii și apelează **interrupt handling routine**, numărul IRQ fiind un index în tabela de întreruperi
- rutina salvează starea, determină cauza întreruperii, procesează, restaurează starea și execută o instrucțiune de întoarcere din întrerupere pentru restaurarea stării CPU dinainte de apariția IRQ
- pentru un sistem modern, este nevoie să se suspende execuția întreruperilor în timpul procesărilor critice, respectiv să distingem între întreruperi high- și low- priority

Anatomia unei întreruperi



Chaining. Nivele de prioritate

- majoritatea CPU-urilor au două linii de IRQ, **nemascabile**, rezervate de ex. evenimentelor de eroare de memorie nerecuperabile, respectiv **mascabile**, care pot fi dezactivate de CPU
- în practică tabela de întreruperi are mai puține intrări decât dispozitive, astfel că se practică **interrupt chaining**, fiecare intrare indică spre o listă de handleri de întreruperi
- mecanismul întreruperilor implementează **nivele de prioritate a întreruperilor**, permițând celor mai prioritare să le "mascheze" pe cele mai puțin prioritare
- întreruperile sunt folosite pentru tratarea evenimentelor asincrone, urgente
- tratarea întreruperilor trebuie să se facă eficient pentru ca sistemul să fie performant

Vectorul de întreruperi de la sistemele Intel

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor**
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului
- 7 Open-source
- 8 Bibliografie

Structura de stocare

- CPU încarcă instrucțiuni din memoria RAM, implementată în tehnologie DRAM
- la pornire memoria RAM e goală, fiind volatilă; calculatorul rulează la pornire **bootstrap program**, boot loader-ul, care încarcă SO
- acesta e menținut într-o memorie programabilă, persistentă (read-only) ce poate fi ștearsă doar electric - EEPROM
- fiecare dispozitiv are de regulă propriul **firmware**, program de funcționare stocat într-un EEPROM
- **arhitectura de calcul von Neumann** este caracterizată de:
încărcarea instrucțiunii din memorie în Instruction Register (IR);
decodarea instrucțiunii ce cauzează aducerea operanzilor din memorie și stocarea în regiștrii interni; execuția instrucțiunii și stocarea în memorie a rezultatelor

Medii de stocare

- memoria RAM este volatilă și insuficientă pentru a stoca toate programele și datele
- memoriile secundare trebuie să poată înmagazina volume mari de date
- **hard-disk drives** (HDD) și **nonvolatile memories** (NVM) sunt folosite pentru înmagazinarea datelor și a programelor, mult mai lente decât memoria principală Random Acces Memory (RAM)

Ierarhia dispozitivelor de stocare

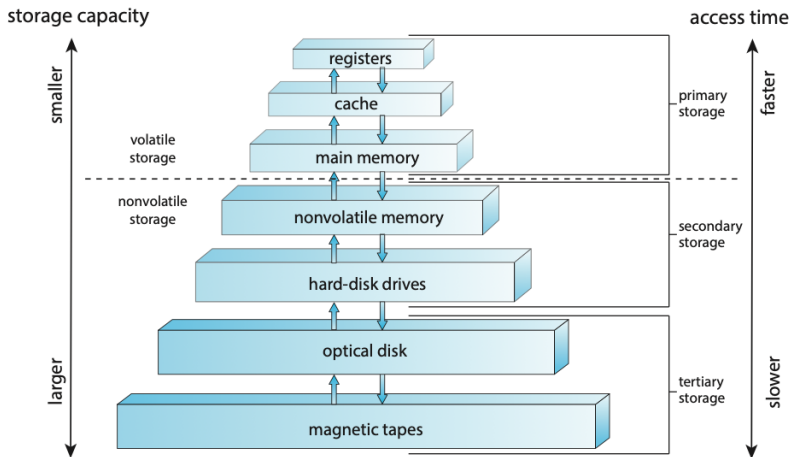
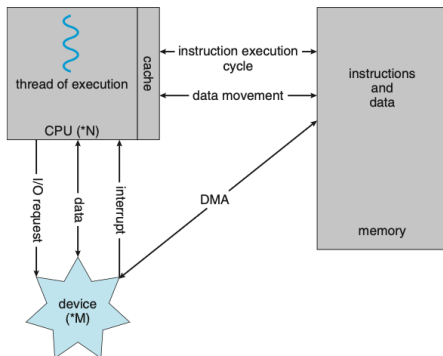


Figure 1.6 Storage-device hierarchy.

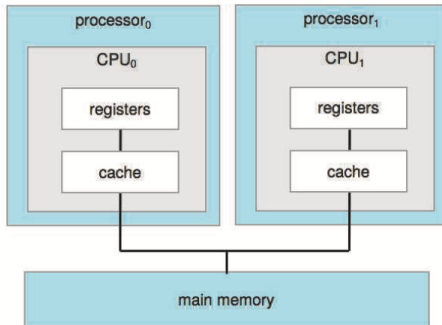
Transferul Direct Memory Access (DMA)

- transferul datelor în volume mari produce un overhead ridicat folosind mecanismul întreruperilor (buffer mic de date pe device)
- transferul DMA presupune setarea bufferelor, pointerilor, counterelor pentru dispozitiv, apoi device controller transferă datele direct în memoria principală, fără intervenția CPU; acesta este liber pentru alte procesări; IRQ se generează doar la terminarea transferului blocului



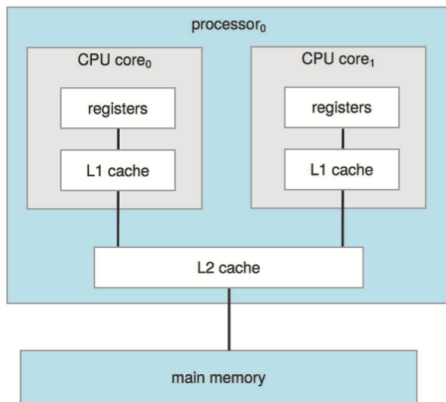
Arhitecturi de tip Symmetric Multi-Processing (SMP)

- CPU-urile conțin mai multe unități de procesare (**core-uri**), ce execută instrucțiunile unui proces
- există unități de execuție și în dispozitive, dar au un set redus de instrucțiuni și execută doar programe specifice
- procesoarele SMP au replicate seturi de registre și de unități de execuție în fiecare core; în teorie, un program ar putea fi executat de două ori mai repede



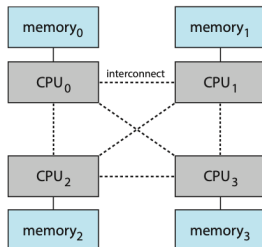
Design de tip dual-core

- CPU-ul are două procesoare, fiecare cu cache-ul său de nivel 1 (date și instrucțiuni)
- cache-ul de tip level 2 este comun ambelor core-uri; mai mare ca L1, dar și mai lent

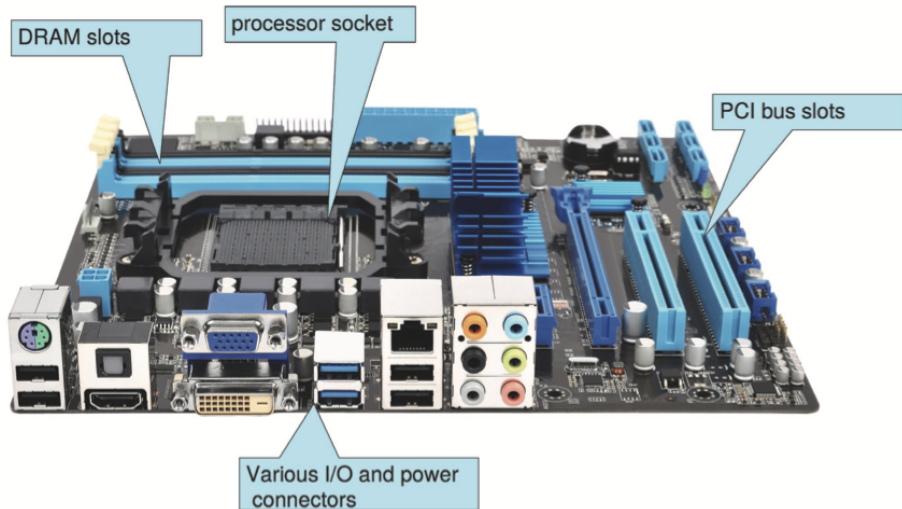


Arhitectura de tip Non-Uniform Memory Acces (NUMA)

- pe măsură ce la un sistem se adaugă mai multe core-uri, acesta nu mai scalează bine din cauza bus-ului care devine bottleneck, iar performanța are de suferit
- alternativa (NUMA) este ca fiecare CPU (core) să aibă propria memorie; CPU-urile sunt legate printr-o rețea de interconectare (ex. CUDA)
- penalizarea apare la accesul memoriei non-locale (ex. CPU₀ accesează memoria lui CPU₃); minimizarea comunicației se face de SO prin planificarea atentă a CPU precum și de alocatorul de memorie



Placa de bază a unui PC



- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO**
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului
- 7 Open-source
- 8 Bibliografie

Cum funcționează SO

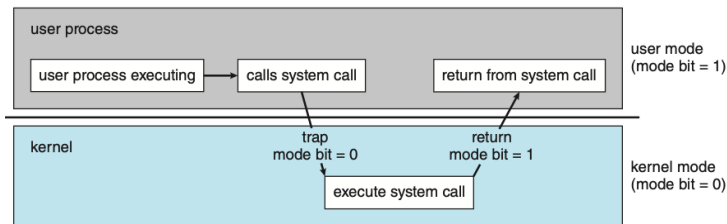
- la punerea sub tensiune, programul de bootstrap stocat în firmware (placa de bază) va inițializa regiștrii tuturor dispozitivelor și încărca în memoria principală kernel-ul SO
- odată încărcat, kernel-ul poate furniza servicii sistemului și utilizatorilor săi
- o parte din serviciile SO sunt furnizate de programe pornite la bootare, care rulează indefinit odată cu kernel-ul; acestea sunt programe **daemon**; primul daemon încărcat este **systemd**, care pornește ceilalți daemoni
- un sistem fără procese de executat și fără evenimente va aștepta să se întâmple ceva, de ex. o întrerupere
- întreruperile pot fi generate și software, denumite **trap** (excepție), de exemplu la apariția împărțirii cu zero sau atunci când un program utilizator solicită un serviciu SO prin execuția unui apel de sistem (**system call**)

Multiprogramarea și multitasking-ul

- un singur program nu poate ține ocupat CPU la 100%, astfel că se rulează mai multe programe cvasi-simultan, prin **multiprogramare**; un program în execuție se numește **proces**
- un proces se execută pentru o durată de timp iar apoi, fie are nevoie ca dispozitivul I/O să termine operația, sau este întrerupt; un alt proces este planificat și lansat în execuție; CPU nu devine idle
- în **multitasking**, CPU execută procesele prin comutarea deasă între ele, iar utilizatorul percepe un **timp de răspuns** mic
- pentru a cuprinde toate procesele, SO utilizează **memoria virtuală**, care permite execuția unui proces care nu se află decât parțial în memorie, uneori poate fi mult mai mare decât **memoria fizică**

Modul de operare dual

- utilizatorii și aplicațiile folosesc în comun resursele sistemului; un SO bine proiectat se va asigura că un program rău-intenționat nu poate afecta SO sau alte programe
- sistemul distinge între execuția codului utilizator și cea a codului SO; HW pune la dispoziție moduri diferite: **user mode** și **kernel mode** (supervizor mode, system mode sau privileged mode)
- HW va permite execuția instrucțiunilor privilegiate doar în modul kernel, altfel se aruncă o excepție (trap)



Modul de operare multiplu

- procesoarele Intel au patru inele de protecție, în care inelul 0 este kernel-ul iar inelul 3 este modul utilizator
- arhitectura ARM (Advanced RISC Machines, de ex. Apple M1) are 7
- CPU-urile care suportă **virtualizare** au un mod separat pentru Virtual Machine Manager (VMM), mod cu mai puține privilegii decât kernel-ul dar mai multe ca user mode
- programul se execută în user mode iar trecerea în kernel mode se face prin system call (trap)
- **system calls** sunt tratate ca întreruperi software; se generează un IRQ iar adresa de tratare se ia din vectorul de întreruperi, după care se dă controlul kernel-ului; acesta verifică parametrii, execută și dă controlul înapoi programului care a generat trap
- dacă programul execută o instrucțiune privilegiată sau accesează o zonă de memorie nealocate lui, HW generează tot o întrerupere; SO va decide ce să facă cu programul invalid (poate restart)

- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO**
- 6 Structurile de date ale kernel-ului
- 7 Open-source
- 8 Bibliografie

Managementul proceselor

- sistemul are un **timer** (cronometru) care generează întreruperi HW la un interval predeterminat (ex. de 250 ori pe secundă, 4 ms)
- controlul este dat în mod regulat SO care întrerupe procesul curent (fără excepție)
- valoarea contorului (HZ) depinde de cum este configurat kernel-ul, precum și de arhitectura mașinii pe care lucrează; variabila kernel se numește **jiffies**, ce reprezintă numărul de câte ori a aparărut acest timer IRQ de la pornirea sistemului
- un proces are nevoie de resurse: timp CPU, memorie fișiere și dispozitive I/O; la terminare, SO va elibera resursele utilizate de el
- un **program** NU este un proces, ci doar o entitate pasivă stocată
- un **proces** single-threaded are asociat un **program counter** ce specifică următoarea instrucțiune ce se va executa; instrucțiunile se execută la rând
- OS e responsabil de crearea și ștergerea proceselor, planificarea, suspendarea, sincronizarea și comunicațiile lor

Management-ul memoriei

- memoria este un array mare de bytes, fiecare cu adresa lui, folosită pentru stocarea datelor și a programelor (arhitectura von Neumann)
- mai multe programe sunt simultan în memorie (parțial), avem nevoie de algoritmi de memory management particulari pentru acea arhitectură
- OS va ține evidența folosirii zonelor de memorie de către procese, alocarea și dealocarea memorie pentru procese, decide ce părți din procese se mută în și din memorie

Management-ul fișierelor

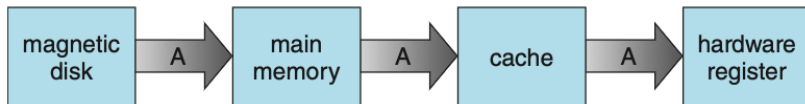
- fișierul este o modalitate uniformă și abstractă de vizibilitate a datelor oferită de SO
- acesta e o colecție de informații asociate definite de creatorul lor - formatate liber, ca text, sau rigid, cu câmpuri fixe precum fișierele MP3
- ele sunt organizate în directoare și li se poate restricționa utilizatorilor accesul la ele (read, write, append)
- SO asigură servicii pentru crearea și ștergerea fișierelor și directoarelor, manipularea lor, maparea fișierelor pe stocarea de masă, backup pe medii non-volatile

Stocarea secundară (mass storage)

- majoritatea programelor sunt stocate pe acestea (HDD, SSD/NVM), sau le folosesc pentru citirea/scrierea rezultatelor
- SO este responsabil pentru mount/unmount, management-ul spațiului, alocare, partiționare, protecție; viteza pentru mediile secundare este crucială pentru performanța sistemului
- stocarea terțiară (CD, DVD, BluRay, banda magnetică) este mai lentă dar mai sigură pe termen lung, tot administrată de SO (aceleași op.)

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

Cache management



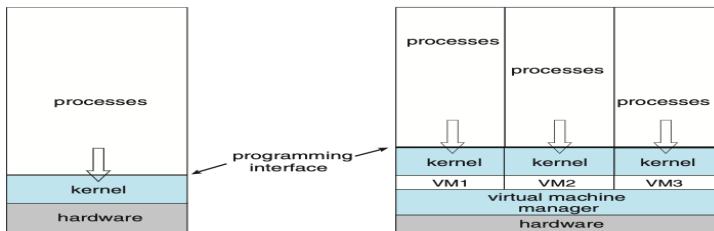
- **cache**-ul este memoria intermediară în care sunt aduse datele din memoria principală (DRAM), temporar; la solicitarea următoare, dacă datele există deja, sunt aduse din cache direct; cache-ul este mai rapid și se încarcă cu mai multe date odată, date ce oricum vor fi citite (ex. program)
- cache-ul este implementat la interfața registri-memorie; este limitat, de aceea pentru performanțe ridicate avem nevoie de algoritmi de alocare a sa
- ex. o valoare A este stocată pe disk și dorim să o incrementăm; ea trece în memoria principală și, prin cache, în regiștri HW, deci apare în mai multe cache-uri simultan; dacă mai multe procese/thread-uri accesează aceeași valoare pentru scriere/citire, avem de rezolvat problema sincronizării accesului la ea (**cache coherency**)

Securitate și protecție

- mai mulți utilizatori rulează mai multe programe concomitent pe sistem; SO se asigură că CPU, fișierele, segmentele de memorie și alte resurse au autorizările necesare ca să nu se perturbe reciproc (accesează propria memorie, nu se blochează în bucle infinite etc.)
- **protecția** este enforșată de SO asupra memoriei și resurselor sale
- **securitatea** protejează SO de atacuri interne sau externe (virusi, viermi, troieni, DoS, furtul identității)
- protecția și securitatea impun identificarea utilizatorilor prin **user identifier** - **userID**, asociind utilizatorul printr-un număr unic cu procesele și fișierele sale
- grupurile de utilizatori se disting printr-un **groupID**, au acces in-corpore la anumite funcționalități; un utilizator poate fi membru a mai multor grupuri; utilizator poate **escala privilegiile** pentru a câștiga permisiuni pentru activități speciale
- procesul poate rula cu **effectiveID**, privilegiile utilizatorului curent, sau avea atributul **setuid**, pentru a rula cu privilegiile creatorului

Virtualizarea

- abstractizarea HW (CPU, memorie, disc NICs etc.) în mai multe medii de execuție, ex. SO individuale, **mașini virtuale**
- virtualizarea este o subclasă a emulării; aceasta e folosită când un program este scris pentru o altă arhitectură decât cea curentă ex. "Rosetta", folosit pentru execuția codului IBM Power CPU (**guest**) pe Intel x86 CPU (**host**), în mașinile Apple
- emularea presupune traducerea fiecărei instrucțiuni de pe o platformă pe alta; codul emulat merge mult mai lent decât codul nativ (ex. e posibil să rulăm cod Intel x86 pe Apple M1 ARM - foarte lent)



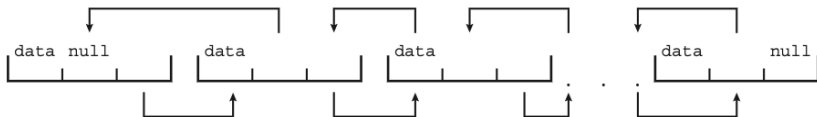
- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului**
- 7 Open-source
- 8 Bibliografie

Structuri de date folosite de kernel: liste, stive, cozi

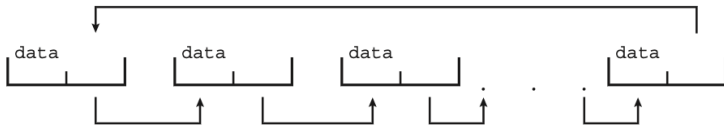
- array: fiecare element se accesează direct, prin index (ex. main memory)



- listă: elemente accesate în ordine (unul după altul); simplu- , dublu-înlănțuită

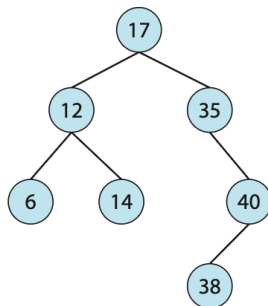


- ... sau circular înlănțuită; stivă (LIFO) vs. coadă (FIFO)



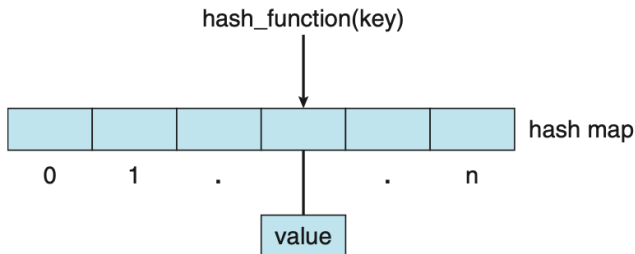
Structuri de date pentru kernel: arbori

- arbore: reprezentare ierarhică a datelor, relație părinte-copil
- arborele binar de căutare (copil stânga $\leq$ copil dreapta) poate avea timpul de căutare în $O(n)$ (de ce?)
- arborii de căutare binari echilibrați conțin N noduri iar înălțimea este în $O(\lg N)$
- Linux folosește un astfel de arbore de tip red-black ca parte a planificatorului CPU



Structuri de date pentru kernel: hash maps

- o **funcție hash** primește ca argument un număr, calculează și returnează alt număr, folosită ca index pentru a accesa date într-o tabelă
- este folosită la căutare deoarece ordinul de timp este foarte bun, $O(1)$
- pentru valori diferite ale intrării funcția hash poate calcula același rezultat - **coliziune hash**; la acea locație putem implementa o listă în care căutăm liniar (de lungime cât mai mică)
- parolele "condimentate" (salted, adăugat caractere random) și apoi hashed, înainte de a fi stocate în baza de date



Structuri de date pentru kernel: bitmaps

- un **bitmap** este un șir de n cifre binare ce reprezintă starea a n componente ale SO (ex. disponibilitatea unei resurse)
- puterea reprezentării vine din eficiența ocupării spațiului - putem ține evidența ocupării a 65536 resurse cu 16 biți
- o unitate de disc de dimensiune medie poate fi împărțită în câteva mii de unități, denumite **blocuri**; un bitmap poate fi folosit pentru a indica disponibilitatea (liber / alocat) pentru fiecare bloc

- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului
- 7 Open-source**
- 8 Bibliografie

Sisteme de operare free și open-source (1)

- free software nu doar pune la dispoziție sursele, ci și permite folosirea, redistribuția și modificarea fără costuri, pe când open-source doar pune la dispoziție sursele
- Linux are distribuții atât free cât și open-source¹
- Microsoft Windows este un sistem **closed-source**, proprietar; Apple macOS are un kernel open-source (Darwin) dar include și componente closed-source; în general companiile sunt reticente în a oferi software-ul open, dar unele precum RedHat fac acest lucru și realizează venituri din suportul oferit utilizatorilor
- în 1984 Richard Stallman a pornit dezvoltarea unui sistem UNIX-like denumit GNU (Gnu's Not Unix), care promovează policy-ul de a utiliza, modifica și revinde/redistribui software-ul, fără costuri
- în 1991 studentul Linus Torvalds a lansat un kernel UNIX-like construit cu uneltele GNU, denumit "Linux", care a adoptat licența de distribuție GNU

¹<http://www.gnu.org/distros/>

Sisteme de operare free și open-source (2)

- BSD-UNIX a pornit în 1978 ca derivat din UNIX-ul AT&T, lansat de University of California at Berkeley; îngreunat de procesele cu AT&T, a apărut ca versiune open-source doar ca versiunea 4.4 BSD în 1994
- atât Linux cât și BSD au multe distribuții; kernel-ul macOS este bazat pe BSD UNIX²
- Solaris este sistemul comercial de tip UNIX al Sun Microsystems, ulterior deschis ca OpenSolaris
- vom folosi software de virtualizare precum VirtualBox sau Qemu pentru a instala și lucra pe distribuția de Linux Fedora 36 Server³

²<http://www.opensource.apple.com/>

³<https://getfedora.org/en/server/download/> sau

<https://mirrors.chroot.ro/fedora/linux/releases/36/Server/>

- 1 Introducere
- 2 Organizarea unui sistem de calcul
- 3 Stocarea și transferul datelor
- 4 Funcționarea unui SO
- 5 Responsabilitățile unui SO
- 6 Structurile de date ale kernel-ului
- 7 Open-source
- 8 Bibliografie**

Bibliografie

- ① Abraham Silberschatz, Peter Baer Galvin, Greg Gagne, *Operating System Concepts*, 10th edition, Wiley 2018
- ② Andrew S. Tanenbaum, Albert S. Woodhull, *Operating Systems: Design and Implementation*, 3rd edition, Prentice Hall 2006
- ③ William Stallings, *Operating Systems: Internals and Design Principles*, 6th edition
- ④ Machtelt Garrels, *Introduction to Linux: A Hands on Guide*, 1.27 edition, 2008
- ⑤ Machtelt Garrels, *Bash Guide for Beginners*, 1.11 edition, 2008
- ⑥ Mendel Cooper, *Advanced Bash-Scripting Guide*, revision 3.0, 2004